МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2

по курсу «Компьютерная графика»

Камера и освещение

Выполнил: Нгуен Шон
Группа: М8О-305БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы — реализовать камеру с возможностью её перемещения и освещение по модели Блинн-Фонга с использованием нескольких типов источников света. В качестве объекта используется цилиндр из предыдущей лабораторной работы (вариант 8).

Программа должна работать в реальном времени и позволять изменять через интерфейс ImGui параметры камеры (позицию, поворот, угол обзора), источников света (позиция, цвет, интенсивность, уникальные параметры типа) и материалов объектов (альбедо, цвет блика, коэффициент блеска).

Выполнены базовое задание и все три дополнительных задания, необходимых для оценки «4»:

- 1. Разделение внешних данных на два набора дескрипторов: для сцены (uniform-буфер + storage-буферы источников) и для модели (uniform-буфер параметров материала).
- 2. Освещение по Блинн-Фонгу массивом точечных источников из storage-буфера с затуханием по закону обратных квадратов, добавление и удаление источников через ImGui.
- 3. Освещение по Блинн-Фонгу массивом прожекторных источников из storage-буфера с затуханием по закону обратных квадратов и мягкими краями освещаемой области (внутренний и внешний углы), добавление и удаление источников через ImGui.

У вершин объекта задан атрибут нормали, нормали генерируются процедурно. Для доступа к массивам источников используются storage-буферы, поэтому максимальное количество элементов в шейдере не задаётся.

Метод решения

Камера

Камера реализована как орбитальная: она вращается вокруг начала координат на расстоянии distance, задаваемом пользователем. Положение камеры вычисляется из двух углов – yaw (горизонтальный) и pitch (вертикальный, ограничен ±85°):

$$camera_pos = target + distance \cdot (\cos(pitch)\cdot\sin(yaw),\ \sin(pitch),\ \cos(pitch)\cdot\cos(yaw))$$

Матрица вида вычисляется через glm::lookAt(camera_pos, target, up), где up = (0, 1, 0). Матрица проекции – перспективная с углом обзора FOV, который также настраивается через ImGui. Для Vulkan выполняется инверсия компоненты proj[1][1] (Y-flip), а для корректной работы ортографической проекции используется макрос GLM_FORCE_DEPTH_ZERO_TO_ONE, заданный до подключения glm.

Нормали вершин

Каждая вершина цилиндра теперь хранит позицию, цвет и нормаль. Нормали генерируются процедурно в функции generateCylinder:

- боковая поверхность – радиальная нормаль (cos α, 0, sin α);
- нижняя крышка – (0, −1, 0);

- верхняя крышка – (0, 1, 0).

В вершинном шейдере нормаль трансформируется матрицей нормалей – inverse transpose от матрицы модели, что корректно работает и при неравномерном масштабе:

*fragNormal = normalize(mat3(transpose(inverse(model))) * inNormal)*

Модель освещения Блинн-Фонга

Освещение считается во фрагментном шейдере по модели Блинн-Фонга. Итоговый цвет – сумма трёх слагаемых:

$$Color = Ambient + \Sigma\ (Diffuse + Specular)$$

Ambient: ambient_light · albedo.

Diffuse: light_color · max(dot(N, L), 0) · albedo.

Specular: light_color · pow(max(dot(N, H), 0), alpha) · specular, где H = normalize(L + V).

Здесь N – нормаль поверхности, L – вектор на источник света, V – вектор на камеру, H – половинчатый вектор, alpha – коэффициент блеска материала. Каждое слагаемое умножается на затухание, если источник точечный или прожекторный.

Источники света

Направленный источник (directional). Задаётся направлением и цветом. Вектор на свет L = normalize(−direction), затухания нет. Параметры хранятся прямо в Scene UBO.

Точечный источник (point). Задаётся позицией и цветом. Вектор L = normalize(position − worldPos), затухание по закону обратных квадратов: attenuation = 1 / max(d^2, ε), где d – расстояние до источника. Массив точечных источников хранится в отдельном storage-буфере (binding 1).

Прожекторный источник (spot). Задаётся позицией, цветом, направлением, внутренним и внешним углами. Затухание такое же, как у точечного, но дополнительно умножается на spotFactor. Угол θ между направлением прожектора и вектором на фрагмент сравнивается с углами: если θ ≤ inner – яркость 1; если θ ≥ outer – 0; между ними – линейный спад. При этом выполняется условие задания: мягкие края, если внешний угол больше внутреннего. Массив прожекторов хранится в отдельном storage-буфере (binding 2).

Дескрипторы и структуры данных

Внешние данные разделены на два набора дескрипторов. Set 0 (сцена) содержит SceneUniforms (матрица view-projection, позиция камеры, параметры направленного света, ambient, количество точечных и прожекторных источников) и два storage-буфера с массивами источников. Set 1 (модель) содержит ModelUniform (матрица модели, альбедо, цвет блика, alpha).

Для корректной передачи данных в std140 в UBO и std430 в storage-буферах поля vec3 в C++ дополнены явными паддингами, чтобы offset полей совпадал с GLSL. Для структур PointLight и Spotlight использован alignas(16), чтобы размер совпадал с GLSL (32 и 64 байта). Это проверяется static_assert при компиляции.

Массивы источников объявлены в GLSL как runtime-sized (без указания максимального количества), доступ к ним осуществляется через readonly buffer. Максимальное количество (16) задано только в C++ для размера аллокации.

Отрисовка и интерфейс

Перед отрисовкой привязывается scene-набор дескрипторов, затем для каждого объекта – свой model-набор. Используется один вершинный и один индексный буфер: цилиндр рисуется дважды с разными матрицами модели, что демонстрирует разделение сцены и модели.

Через ImGui доступны: камера (yaw, pitch, distance, FOV), ambient, направленный свет (направление, цвет), параметры каждого объекта (позиция, поворот, масштаб, альбедо, цвет блика, alpha), списки точечных и прожекторных источников с кнопками добавления, удаления и полями всех параметров.

Результаты

Ниже приведены снимки работающей программы. В сцене два цилиндра с разными материалами, освещённые направленным светом, точечным источником и прожектором.

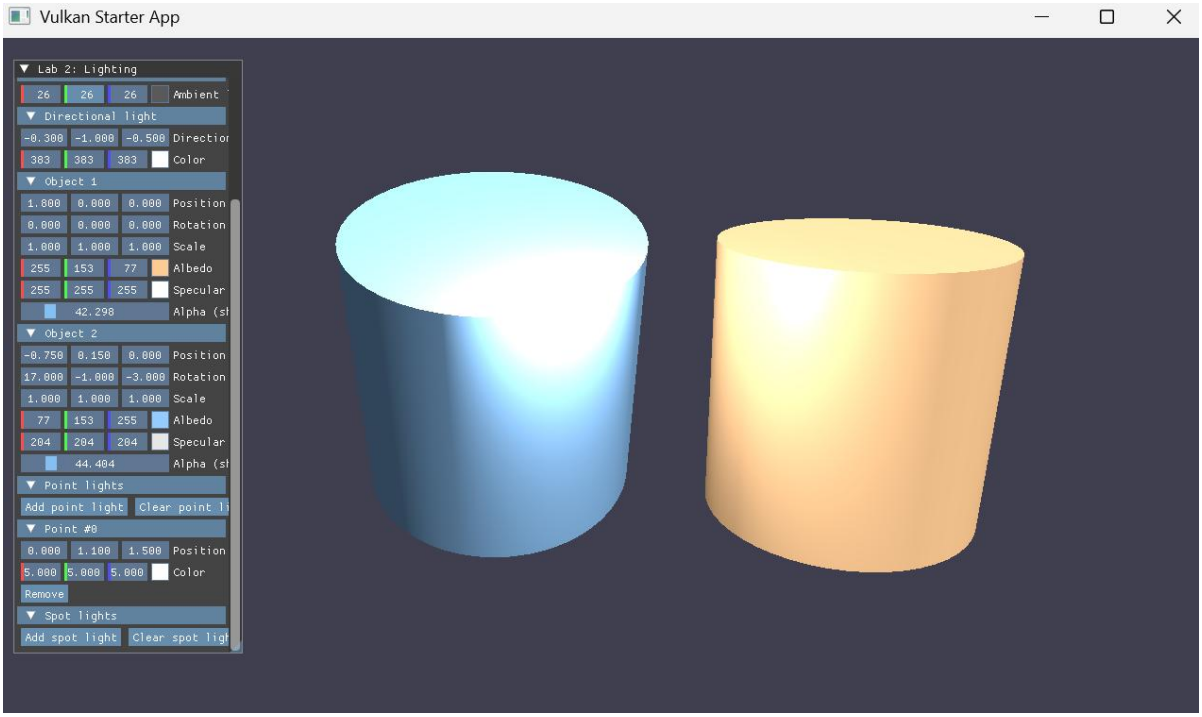


Рисунок 1 – Направленный и точечный свет

На цилиндрах виден градиент от направленного света (светлая и тёмная сторона) и локальное яркое пятно от точечного источника с затуханием по закону обратных квадратов.

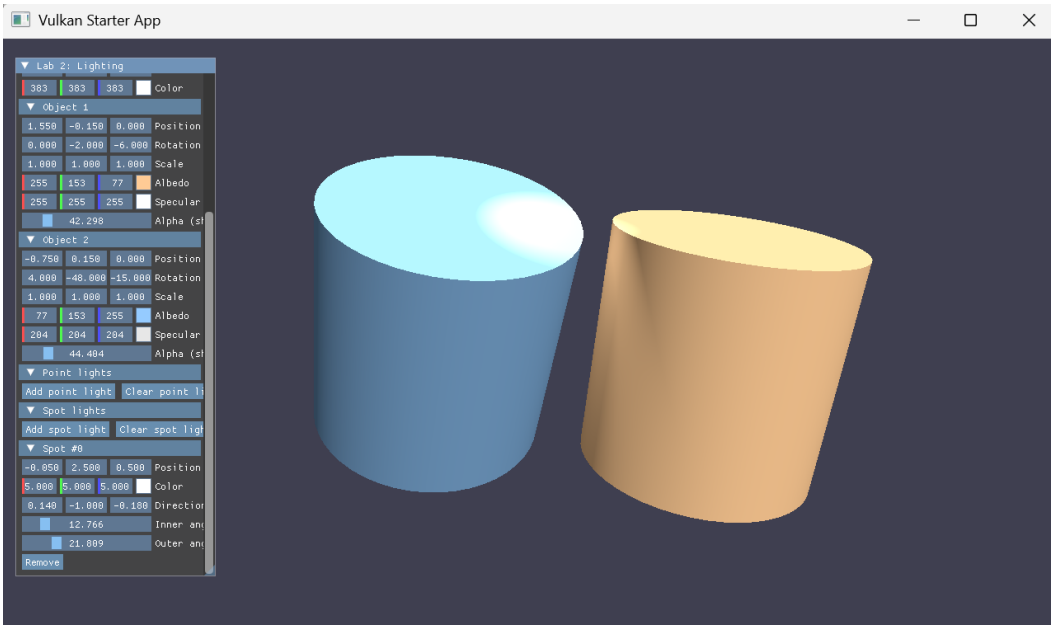


Рисунок 2 – Прожекторный источник с мягкими краями

Прожектор создаёт ограниченную освещённую область. За внешним углом света нет, между внутренним и внешним углами яркость плавно спадает. При изменении углов в ImGui размер и мягкость края меняются.

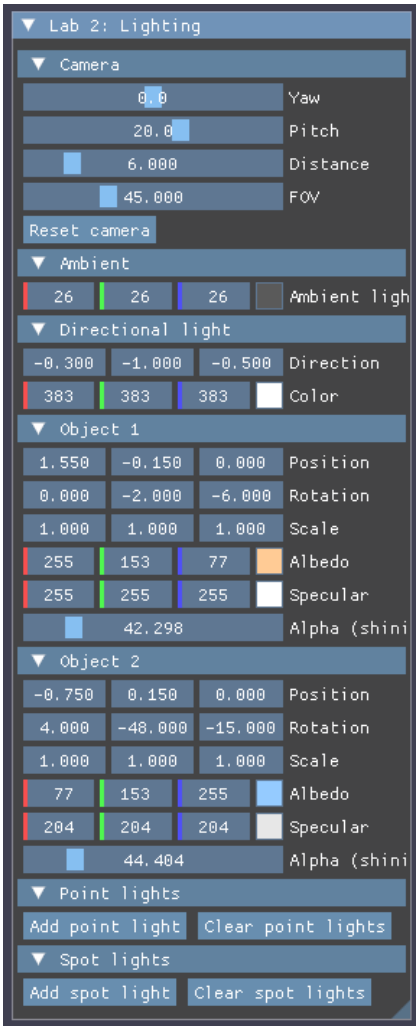


Рисунок 3 – Интерфейс ImGui

Через ImGui настраиваются все параметры: камера, ambient, направленный свет, материалы обоих объектов, а также добавление, удаление и параметры точечных и прожекторных источников.

Выводы

В работе я реализовал орбитальную камеру с настройкой положения, поворота и угла обзора и освещение по модели Блинн-Фонга с тремя типами источников: направленным, точечным и прожекторным. Для точечного и прожекторного источников реализовано затухание по закону обратных квадратов, для прожекторного – мягкие края через внутренний и внешний углы.

Нормали вершин генерируются процедурно и трансформируются матрицей нормалей (inverse transpose от матрицы модели). Внешние данные разделены на два набора дескрипторов: сцена (uniform-буфер + storage-буферы источников) и модель (uniform-буфер материала). Массивы источников хранятся в storage-буферах без указания максимального размера в шейдере.

Я научился работать с несколькими наборами дескрипторов, storage-буферами переменного размера, вычислять освещение во фрагментном шейдере, корректно выравнивать структуры под std140/std430 и настраивать все параметры сцены через ImGui.